American International University – Bangladesh

Faculty of Science and Technology

Introduction To Data Science

Section- E, Group- C

Session: Fall 2025-26

Dataset Title

Premier League All Players Stats 23/24

Supervised by

KAMRUN NAHER KOLI

Group Member:

Name	ID
Arafat Hossain	23-50277-1
Symon Islam Parash	22-47340-2
Tohidul Islam Khan	22-49658-3

Premier League All Players Stats 23/24

For the midterm project of "Introduction to Data Science" course we have collected a dataset which contains detailed data on all footballers from the 2023/24 premier League. Source: https://www.kaggle.com/datasets/orkunaktas/premier-league-all-players-stats-2324

1. Dataset Description:

More details about the dataset-

- Player: The name of the player.
- Nation: The player's nationality.
- Pos: The player's position (e.g., forward, midfielder, defender).
- Age: The player's age.
- MP (Minutes Played): Total minutes played by the player.
- Starts: Number of matches the player started.
- Min (Minutes): Total minutes played by the player (this might be the same as MP).
- 90s (90s Played): The equivalent of 90-minute matches played by the player (e.g., 1.5 = 135 minutes).
- Gls (Goals): Total number of goals scored by the player.
- Ast (Assists): Total number of assists made by the player.
- G+A (Goals + Assists): Total number of goals and assists combined.
- G-PK (Goals - Penalty Kicks): Total number of goals scored excluding penalty kicks.
- PK (Penalty Kicks): Number of penalty goals scored by the player.
- PKatt (Penalty Kicks Attempted): Number of penalty kicks attempted by the player.
- CrdY (Yellow Cards): Number of yellow cards received by the player.
- CrdR (Red Cards): Number of red cards received by the player.
- xG (Expected Goals): The expected number of goals from the player's shots.
- npxG (Non-Penalty Expected Goals): Expected goals excluding penalties.
- xAG (Expected Assists): The expected number of assists from the player's passes.
- npxG+xAG (Non-Penalty xG + xAG): Total of non-penalty expected goals and expected assists.
- PrgC (Progressive Carries): Number of times the player carried the ball forward.
- PrgP (Progressive Passes): Number of passes made by the player that moved the ball forward.

- PrgR (Progressive Runs): Number of times the player made runs forward with the ball.
- G+A-PK (Goals + Assists - Penalty Kicks): Total goals and assists minus penalty goals.
- xG+xAG (Expected Goals + Expected Assists): Total expected goals and assists.
- npxG+xAG (Non-Penalty xG + Expected Assists): Total of non-penalty expected goals and expected assists.

2. Data Issues Introduced:

As we are working with a secondary dataset, we intentionally introduce a few data issues like missing values, incorrect entries, inconsistent formats, duplicate rows etc.

- **Missing values:** row 27,42,151,203,287 of column: Age, row 97,199,352,447,510 of column: MP and row 16,223,390,596,534 have missing values.
- **Incorrect entries:** row 16,124,234 of column: Age and row 204,309 of column: MP have some incorrect entries
- **Inconsistent formats:** row 72 of column: Pos has inconsistent format like midfielder instead of MF
- **duplicate rows:** row 548 and 572 have been duplicated

3. Data Import in R:

```
> library(readxl)
```

```
Warning message:
package 'readxl' was built under R version 4.5.2
```

```
> pl <- read_excel("C:/Users/ahfgc/OneDrive/Desktop/DS_dataset/premier-player-23-
24 - Copy.xlsx")
> head(pl)
# A tibble: 6 × 34
  Player  Nation Pos     Age    MP Starts   Min `90s`   Gls   Ast `G+A` `G-PK`
  <chr>   <chr>  <chr> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
1 "Rodri" es ESP MF       27    34     34  2931  32.6     8     9    17      8
2 "Phil … eng E… FW,MF    23    35     33  2857  31.7    19     8    27     19
3 "Eders… br BRA GK       29    33     33  2785  30.9     0     0     0      0
4 "JuliÃ… ar ARG MF,FW    23    36     31  2647  29.4    11     8    19      9
5 "Kyle … eng E… DF       33    32     30  2767  30.7     0     4     4      0
6 "Berna… pt POR MF,FW    28    33     29  2578  28.6     6     9    15      6
# ℹ 22 more variables: PK <dbl>, PKatt <dbl>, CrdY <dbl>, CrdR <dbl>,
#   xG <dbl>, npxG <dbl>, xAG <dbl>, `npxG+xAG` <dbl>, PrgC <dbl>,
#   PrgP <dbl>, PrgR <dbl>, Gls_90 <dbl>, Ast_90 <dbl>, `G+A_90` <dbl>,
#   `G-PK_90` <dbl>, `G+A-PK_90` <dbl>, xG_90 <dbl>, xAG_90 <dbl>,
#   `xG+xAG_90` <dbl>, npxG_90 <dbl>, `npxG+xAG_90` <dbl>, Team <chr>
```

Interpret: showing the first 6 row of the dataset to confirm.

```
> str(pl)
tibble [582 × 34] (S3: tbl_df/tbl/data.frame)
 $ Player     : chr [1:582] "Rodri" "Phil Foden" "Ederson" "JuliÃ¡n Ã\u0081lvare
z" ...
 $ Nation     : chr [1:582] "es ESP" "eng ENG" "br BRA" "ar ARG" ...
 $ Pos        : chr [1:582] "MF" "FW,MF" "GK" "MF,FW" ...
 $ Age        : num [1:582] 27 23 29 23 33 28 23 26 28 21 ...
 $ MP         : num [1:582] 34 35 33 36 32 33 31 30 30 28 ...
 $ Starts     : num [1:582] 34 33 33 31 30 29 29 28 28 26 ...
 $ Min        : num [1:582] 2931 2857 2785 2647 2767 ...
 $ 90s        : num [1:582] 32.6 31.7 30.9 29.4 30.7 28.6 28.4 28.4 27.9 25.9 ...
 $ Gls        : num [1:582] 8 19 0 11 0 6 27 0 2 4 ...
 $ Ast        : num [1:582] 9 8 0 8 4 9 5 0 0 1 ...
 $ G+A        : num [1:582] 17 27 0 19 4 15 32 0 2 5 ...
 $ G-PK       : num [1:582] 8 19 0 9 0 6 20 0 2 4 ...
 $ PK         : num [1:582] 0 0 0 2 0 0 7 0 0 0 ...
 $ PKatt      : num [1:582] 0 0 0 2 0 0 8 0 0 0 ...
 $ CrdY       : num [1:582] 8 2 5 2 2 8 1 0 4 3 ...
 $ CrdR       : num [1:582] 1 0 0 0 0 0 0 0 1 0 ...
 $ xG         : num [1:582] 4.1 10.3 0 13 0.4 3.7 29.2 1.4 1.9 3.1 ...
 $ npxG       : num [1:582] 4.1 10.3 0 11.5 0.4 3.7 22.9 1.4 1.9 3.1 ...
 $ xAG        : num [1:582] 3.9 8.4 0.1 6.4 2.6 7.6 4.3 0.3 0.5 1.4 ...
 $ npxG+xAG   : num [1:582] 8 18.7 0.1 17.9 3 11.3 27.2 1.7 2.4 4.5 ...
 $ PrgC       : num [1:582] 76 93 0 64 74 140 35 34 46 63 ...
 $ PrgP       : num [1:582] 376 168 4 103 157 177 26 173 148 136 ...
 $ PrgR       : num [1:582] 55 269 0 180 172 260 126 16 36 115 ...
 $ Gls_90     : num [1:582] 0.25 0.6 0 0.37 0 0.21 0.95 0 0.07 0.15 ...
 $ Ast_90     : num [1:582] 0.28 0.25 0 0.27 0.13 0.31 0.18 0 0 0.04 ...
 $ G+A_90     : num [1:582] 0.52 0.85 0 0.65 0.13 0.52 1.13 0 0.07 0.19 ...
 $ G-PK_90    : num [1:582] 0.25 0.6 0 0.31 0 0.21 0.71 0 0.07 0.15 ...
 $ G+A-PK_90  : num [1:582] 0.52 0.85 0 0.58 0.13 0.52 0.88 0 0.07 0.19 ...
 $ xG_90      : num [1:582] 0.12 0.33 0 0.44 0.01 0.13 1.03 0.05 0.07 0.12 ...
 $ xAG_90     : num [1:582] 0.12 0.26 0 0.22 0.09 0.27 0.15 0.01 0.02 0.06 ...
 $ xG+xAG_90  : num [1:582] 0.24 0.59 0 0.66 0.1 0.4 1.18 0.06 0.09 0.18 ...
```

Interpret: showing the structure of the dataset.

4. Data cleaning:

I. Handling missing values

```
> colSums(is.na(pl))
     Player     Nation        Pos        Age         MP     Starts
          0          0          0          5          5          0
        Min        90s        Gls        Ast        G+A      G-PK
          0          0          0          5          0          0
         PK      PKatt      CrdY      CrdR         xG      npxG
          0          0          0          0          0          0
        xAG   npxG+xAG      PrgC      PrgP       PrgR    Gls_90
          0          0          0          0          0          0
    Ast_90     G+A_90    G-PK_90  G+A-PK_90      xG_90    xAG_90
          0          0          0          0          0          0
 xG+xAG_90    npxG_90 npxG+xAG_90       Team
          0          0          0          0
```

Interpretation: This code counts the missing values in each column which helps us to identify which column needs cleaning. As we intentionally introduced missing values in Age, MP and Ast column, it is showing us the count of NA in those columns.

```
> pl$Age[is.na(pl$Age)] <- mean(pl$Age, na.rm = TRUE)
> pl <- pl[!is.na(pl$MP), ]
> pl <- pl[!is.na(pl$Ast), ]
> colSums(is.na(pl))
     Player      Nation         Pos         Age          MP      Starts
          0           0           0           0           0           0
        Min         90s         Gls         Ast         G+A        G-PK
          0           0           0           0           0           0
         PK       PKatt        CrdY        CrdR          xG        npxG
          0           0           0           0           0           0
        xAG    npxG+xAG        PrgC        PrgP        PrgR      Gls_90
          0           0           0           0           0           0
     Ast_90      G+A_90     G-PK_90   G+A-PK_90       xG_90      xAG_90
          0           0           0           0           0           0
  xG+xAG_90     npxG_90 npxG+xAG_90        Team
          0           0           0           0
```

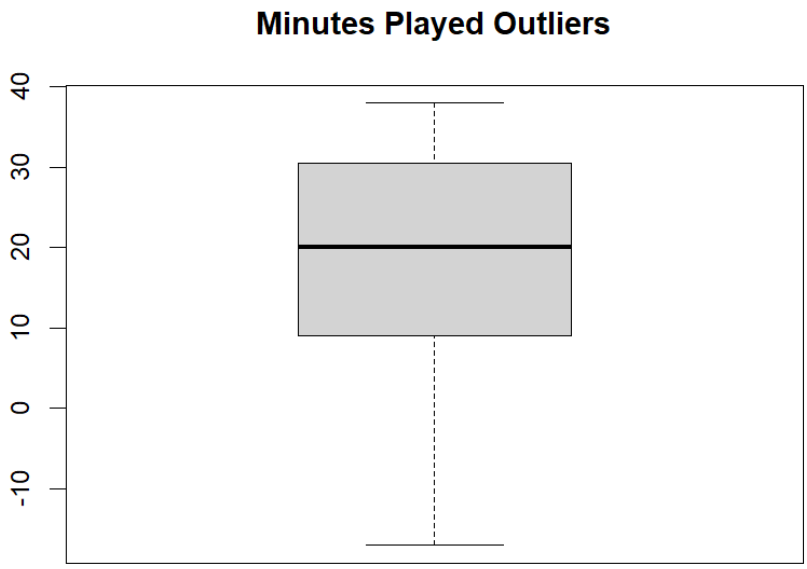
Interpret: we can handle missing values in two ways, either discard instances or replace by mean value. For Age column, missing age were replaced using the mean. For MP and Ast column, missing data was handled by discard the instances. Finally, showing that all missing values were handled by showing NA count.

II. Outlier handling

Code: boxplot(pl$Age, main="Age Outliers")

Interpret: The above boxplot shows that there are some extreme outliers like around 180 and 290 which are not real.

Code: boxplot(pl$MP, main="Minutes Played Outliers")



Interpret: The above boxplot shows that there are some outliers like values in negative which is not possible as minutes played can be zero for lowest.

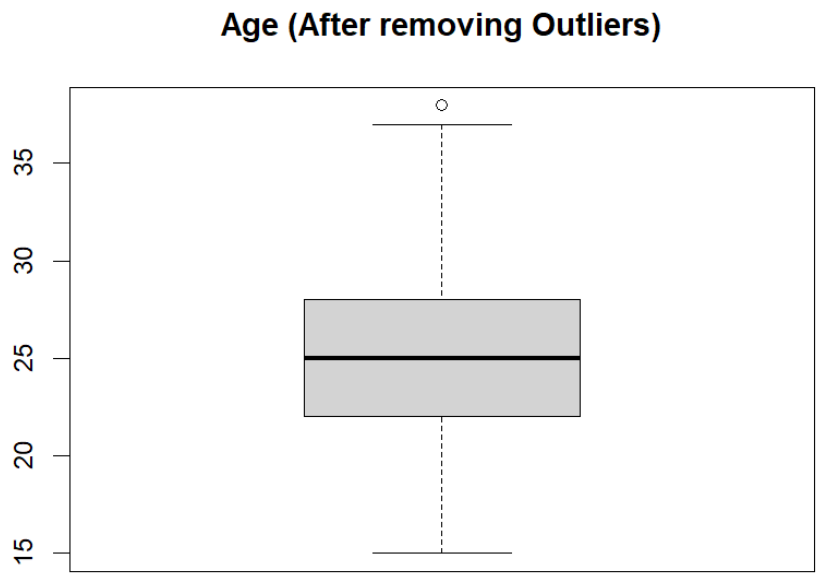
```
> subset(pl, Age < 15 | Age > 45)
# A tibble: 3 × 34
  Player   Nation Pos     Age    MP Starts   Min `90s`   Gls   Ast `G+A` `G-PK`
  <chr>    <chr>  <chr> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
1 John St… eng E… DF,MF   290    16     12  1064  11.8     1     0     1      1
2 Lewis M… eng E… MF      177    17     14  1202  13.4     1     3     4      1
3 Kaine K… eng E… MF,DF     2     3      0     6   0.1     0     0     0      0
# ℹ 22 more variables: PK <dbl>, PKatt <dbl>, CrdY <dbl>, CrdR <dbl>,
#   xG <dbl>, npxG <dbl>, xAG <dbl>, `npxG+xAG` <dbl>, PrgC <dbl>, PrgP <dbl>,
#   PrgR <dbl>, Gls_90 <dbl>, Ast_90 <dbl>, `G+A_90` <dbl>, `G-PK_90` <dbl>,
#   `G+A-PK_90` <dbl>, xG_90 <dbl>, xAG_90 <dbl>, `xG+xAG_90` <dbl>,
#   npxG_90 <dbl>, `npxG+xAG_90` <dbl>, Team <chr>
> subset(pl, MP < 0)
# A tibble: 2 × 34
  Player   Nation Pos     Age    MP Starts   Min `90s`   Gls   Ast `G+A` `G-PK`
  <chr>    <chr>  <chr> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
1 Donny v… nl NED DF       26    -9      0     3   0       0     0     0      0
2 Adama T… es ESP FW,MF    27   -17      1   377   4.2     2     3     5      2
# ℹ 22 more variables: PK <dbl>, PKatt <dbl>, CrdY <dbl>, CrdR <dbl>,
#   xG <dbl>, npxG <dbl>, xAG <dbl>, `npxG+xAG` <dbl>, PrgC <dbl>, PrgP <dbl>,
#   PrgR <dbl>, Gls_90 <dbl>, Ast_90 <dbl>, `G+A_90` <dbl>, `G-PK_90` <dbl>,
#   `G+A-PK_90` <dbl>, xG_90 <dbl>, xAG_90 <dbl>, `xG+xAG_90` <dbl>,
#   npxG_90 <dbl>, `npxG+xAG_90` <dbl>, Team <chr>
```

Interpret: From the boxplot maximum age is between 15 to 45 and MP can't be less than zero. So this code ensures the exact rows that has outlier.

```
> pl <- pl[!(pl$Age < 15 | pl$Age > 45), ]
> pl <- pl[pl$MP >= 0, ]
> subset(pl, Age < 15 | Age > 45)
# A tibble: 0 × 34
# ℹ 34 variables: Player <chr>, Nation <chr>, Pos <chr>, Age <dbl>, MP <dbl>,
#   Starts <dbl>, Min <dbl>, 90s <dbl>, Gls <dbl>, Ast <dbl>, G+A <dbl>,
#   G-PK <dbl>, PK <dbl>, PKatt <dbl>, CrdY <dbl>, CrdR <dbl>, xG <dbl>,
#   npxG <dbl>, xAG <dbl>, npxG+xAG <dbl>, PrgC <dbl>, PrgP <dbl>, PrgR <dbl>,
#   Gls_90 <dbl>, Ast_90 <dbl>, G+A_90 <dbl>, G-PK_90 <dbl>, G+A-PK_90 <dbl>,
#   xG_90 <dbl>, xAG_90 <dbl>, xG+xAG_90 <dbl>, npxG_90 <dbl>,
#   npxG+xAG_90 <dbl>, Team <chr>
> subset(pl, MP < 0)
# A tibble: 0 × 34
# ℹ 34 variables: Player <chr>, Nation <chr>, Pos <chr>, Age <dbl>, MP <dbl>,
#   Starts <dbl>, Min <dbl>, 90s <dbl>, Gls <dbl>, Ast <dbl>, G+A <dbl>,
#   G-PK <dbl>, PK <dbl>, PKatt <dbl>, CrdY <dbl>, CrdR <dbl>, xG <dbl>,
#   npxG <dbl>, xAG <dbl>, npxG+xAG <dbl>, PrgC <dbl>, PrgP <dbl>, PrgR <dbl>,
#   Gls_90 <dbl>, Ast_90 <dbl>, G+A_90 <dbl>, G-PK_90 <dbl>, G+A-PK_90 <dbl>,
#   xG_90 <dbl>, xAG_90 <dbl>, xG+xAG_90 <dbl>, npxG_90 <dbl>,
#   npxG+xAG_90 <dbl>, Team <chr>
```

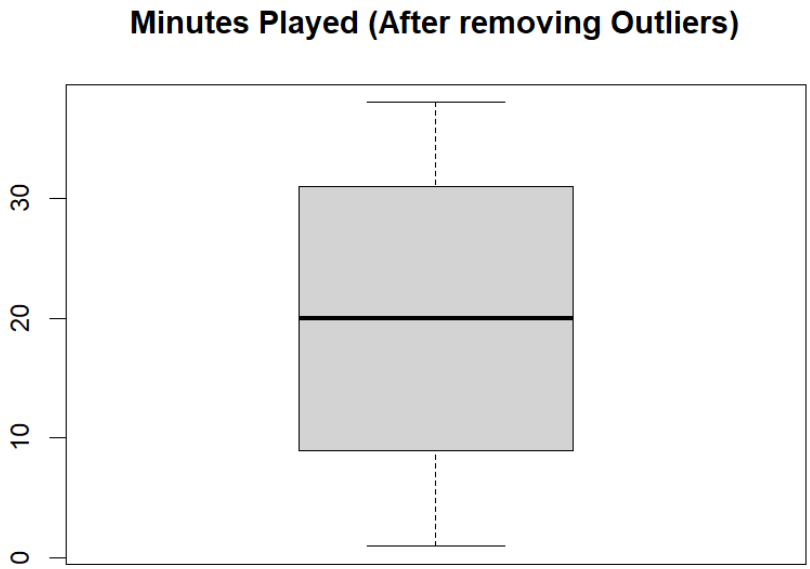
Interpret: We remove the rows that have outliers. Lastly, showing that outlier has been removed using subset.

Code: boxplot(pl$Age, main="Age (After removing Outliers)")



Interpret: visual representation after removing outliers.

Code: boxplot(pl$MP, main="Minutes Played (After removing Outliers)")



Interpret: visual representation after removing outliers.

III. Removing duplicates

```
> sum(duplicated(pl))
[1] 2
> pl <- pl[!duplicated(pl), ]
> sum(duplicated(pl))
[1] 0
```

Interpret: This code shows the number of duplicate rows. Then we removed the duplicate rows.

IV. Fixing inconsistent formats

```
> invalid_pos <- pl[!(pl$Pos %in% c("FW","MF","DF","GK","FW,MF","MF,FW","MF,D
F","DF,FW","FW,DF","DF,MF")), ]
> invalid_pos
# A tibble: 1 × 34
  Player   Nation Pos     Age    MP Starts   Min `90s`   Gls   Ast `G+A` `G-PK`
  <chr>    <chr>  <chr> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
1 Jorginho it ITA midf…  25.6    24     10   921  10.2     0     2     2      0
# i 22 more variables: PK <dbl>, PKatt <dbl>, CrdY <dbl>, CrdR <dbl>,
#   xG <dbl>, npxG <dbl>, xAG <dbl>, `npxG+xAG` <dbl>, PrgC <dbl>, PrgP <dbl>,
#   PrgR <dbl>, Gls_90 <dbl>, Ast_90 <dbl>, `G+A_90` <dbl>, `G-PK_90` <dbl>,
#   `G+A-PK_90` <dbl>, xG_90 <dbl>, xAG_90 <dbl>, `xG+xAG_90` <dbl>,
#   npxG_90 <dbl>, `npxG+xAG_90` <dbl>, Team <chr>
```

Interpret: with this code we have detected an inconsistent format where one position is labeled as midfielder instead of MF.

```
> pl$Pos[pl$Pos == "midfielder"] <- "MF"
> invalid_pos <- pl[!(pl$Pos %in% c("FW","MF","DF","GK","FW,MF","MF,FW","MF,D
F","DF,FW","FW,DF","DF,MF")), ]
> invalid_pos
# A tibble: 0 × 34
# i 34 variables: Player <chr>, Nation <chr>, Pos <chr>, Age <dbl>, MP <dbl>,
#   Starts <dbl>, Min <dbl>, 90s <dbl>, Gls <dbl>, Ast <dbl>, G+A <dbl>,
#   G-PK <dbl>, PK <dbl>, PKatt <dbl>, CrdY <dbl>, CrdR <dbl>, xG <dbl>,
#   npxG <dbl>, xAG <dbl>, npxG+xAG <dbl>, PrgC <dbl>, PrgP <dbl>, PrgR <dbl>,
#   Gls_90 <dbl>, Ast_90 <dbl>, G+A_90 <dbl>, G-PK_90 <dbl>, G+A-PK_90 <dbl>,
#   xG_90 <dbl>, xAG_90 <dbl>, xG+xAG_90 <dbl>, npxG_90 <dbl>,
#   npxG+xAG_90 <dbl>, Team <chr>
```

Interpret: We corrected the label to standard form- MF.

V. Data type conversion

Code: pl$Pos <- as.factor(pl$Pos)

levels(pl$Pos)

Output: [1] "DF" "DF,FW" "DF,MF" "FW" "FW,DF" "FW,MF" "GK" "MF" "MF,DF"

[10] "MF,FW"

Interpret: Position were converted to factor variables as it represents category.

Code: pl$AgeGroup <- cut(

pl$Age,

breaks = c(0, 23, 30, 40),

labels = c("Young", "Prime", "Veteran"),

right = TRUE)

levels(pl$AgeGroup)

Output: [1] "Young" "Prime" "Veteran"

Interpret: age was grouped into “young, prime and veteran” age groups to allow comparative analysis.

VI. Normalization

```
> pl$MP_norm <- (pl$MP - min(pl$MP)) / (max(pl$MP) - min(pl$MP))
> pl$xG_norm  <- (pl$xG - min(pl$xG)) / (max(pl$xG) - min(pl$xG))
> pl$xAG_norm <- (pl$xAG - min(pl$xAG)) / (max(pl$xAG) - min(pl$xAG))
```

Interpret: MP, xG and xAG were normalized using Min Max scaling to bring all values within the range of 0 to 1.

```
> head(pl[, c("MP", "MP_norm")])
# A tibble: 6 × 2
     MP MP_norm
  <dbl>   <dbl>
1    34   0.892
2    35   0.919
3    33   0.865
4    36   0.946
5    32   0.838
6    33   0.865
> head(pl[, c("xG", "xG_norm")])
# A tibble: 6 × 2
     xG xG_norm
  <dbl>   <dbl>
1   4.1   0.140
2  10.3   0.353
3   0     0
4  13     0.445
5   0.4   0.0137
6   3.7   0.127
> head(pl[, c("xAG", "xAG_norm")])
# A tibble: 6 × 2
    xAG xAG_norm
  <dbl>    <dbl>
1   3.9   0.331
2   8.4   0.712
3   0.1   0.00847
4   6.4   0.542
5   2.6   0.220
6   7.6   0.644
```

Interpret: Also showing first few rows comparing original and normalized value of these columns.

VII. Filtering:

Code: prime_players <- pl[pl$Age >= 23 & pl$Age <= 28,]

```
> nrow(prime_players)
[1] 264
> head(prime_players)
# A tibble: 6 × 38
  Player   Nation Pos     Age    MP Starts   Min `90s`   Gls   Ast `G+A` `G-PK`
  <chr>    <chr>  <fct> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
1 "Rodri"  es ESP MF       27    34     34  2931  32.6     8     9    17      8
2 "Phil F… eng E… FW,MF    23    35     33  2857  31.7    19     8    27     19
3 "JuliÃ¡… ar ARG MF,FW    23    36     31  2647  29.4    11     8    19      9
4 "Bernar… pt POR MF,FW    28    33     29  2578  28.6     6     9    15      6
5 "Erling… no NOR FW       23    31     29  2552  28.4    27     5    32     20
6 "RÃºben… pt POR DF       26    30     28  2559  28.4     0     0     0      0
# ℹ 26 more variables: PK <dbl>, PKatt <dbl>, CrdY <dbl>, CrdR <dbl>,
#   xG <dbl>, npxG <dbl>, xAG <dbl>, `npxG+xAG` <dbl>, PrgC <dbl>, PrgP <dbl>,
#   PrgR <dbl>, Gls_90 <dbl>, Ast_90 <dbl>, `G+A_90` <dbl>, `G-PK_90` <dbl>,
#   `G+A-PK_90` <dbl>, xG_90 <dbl>, xAG_90 <dbl>, `xG+xAG_90` <dbl>,
#   npxG_90 <dbl>, `npxG+xAG_90` <dbl>, Team <chr>, AgeGroup <fct>,
#   MP_norm <dbl>, xG_norm <dbl>, xAG_norm <dbl>
```

Code: forwards <- pl[pl$Pos == "FW",]

```
> nrow(forwards)
[1] 91
> head(forwards)
# A tibble: 6 × 38
  Player   Nation Pos     Age    MP Starts   Min `90s`   Gls   Ast `G+A` `G-PK`
  <chr>    <chr>  <fct> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
1 Erling … no NOR FW       23    31     29  2552  28.4    27     5    32     20
2 Luis DÃ… co COL FW       26    37     32  2629  29.2     8     5    13      8
3 Mohamed… eg EGY FW       31    32     28  2534  28.2    18    10    28     13
4 Darwin … uy URU FW       24    36     22  2047  22.7    11     8    19     11
5 Diogo J… pt POR FW       26    21     14  1145  12.7    10     3    13     10
6 Ben Doak sct S… FW       17     1      0    14   0.2     0     0     0      0
# i 26 more variables: PK <dbl>, PKatt <dbl>, CrdY <dbl>, CrdR <dbl>,
#   xG <dbl>, npxG <dbl>, xAG <dbl>, `npxG+xAG` <dbl>, PrgC <dbl>, PrgP <dbl>,
#   PrgR <dbl>, Gls_90 <dbl>, Ast_90 <dbl>, `G+A_90` <dbl>, `G-PK_90` <dbl>,
#   `G+A-PK_90` <dbl>, xG_90 <dbl>, xAG_90 <dbl>, `xG+xAG_90` <dbl>,
#   npxG_90 <dbl>, `npxG+xAG_90` <dbl>, Team <chr>, AgeGroup <fct>,
#   MP_norm <dbl>, xG_norm <dbl>, xAG_norm <dbl>
```

Interpret: this code is to show various filtering techniques, such as selecting prime players whose ages are between 23 to 28. And selecting only the forward players.

5. Exploratory Data Analysis

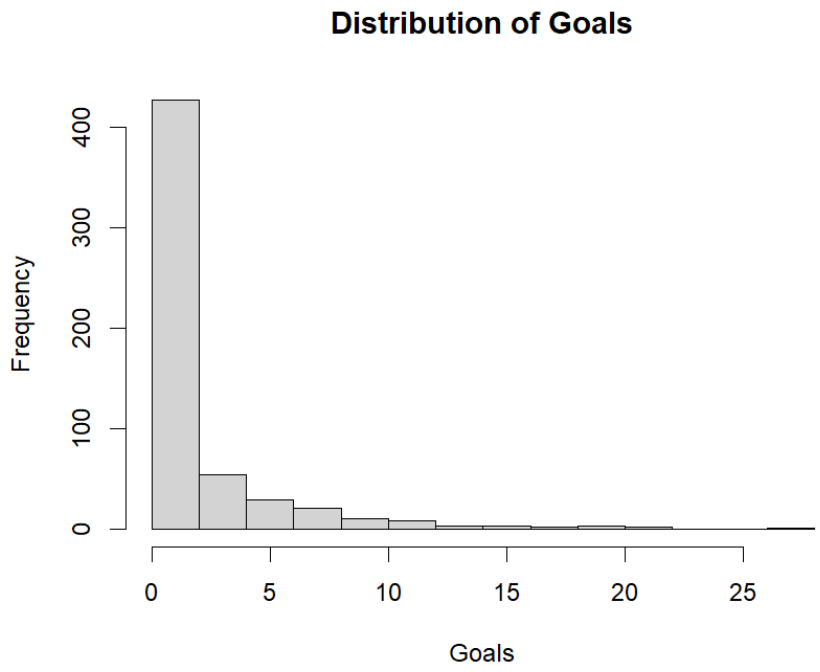
A. Descriptive statistics:

```
> summary(pl[, c("Gls", "Ast", "MP", "xG", "xAG")])
      Gls              Ast              MP              xG
 Min.   : 0.000   Min.   : 0.000   Min.   : 1.00   Min.   : 0.000
 1st Qu.: 0.000   1st Qu.: 0.000   1st Qu.: 9.00   1st Qu.: 0.100
 Median : 1.000   Median : 0.000   Median :20.00   Median : 0.700
 Mean   : 2.053   Mean   : 1.469   Mean   :19.68   Mean   : 2.067
 3rd Qu.: 2.000   3rd Qu.: 2.000   3rd Qu.:31.00   3rd Qu.: 2.500
 Max.   :27.000   Max.   :13.000   Max.   :38.00   Max.   :29.200
      xAG
 Min.   : 0.000
 1st Qu.: 0.100
 Median : 0.700
 Mean   : 1.491
 3rd Qu.: 2.000
 Max.   :11.800
> sd(pl$Gls)
[1] 3.640398
> sd(pl$Ast)
[1] 2.356787
> sd(pl$MP)
[1] 11.89197
> sd(pl$xG)
[1] 3.4229
> sd(pl$xAG)
[1] 2.118659
```

Interpret: The summary statistics show the central tendency and distribution of the above variables. For example, mean of MP indicate average playing time. Range of Gls show the difference between low and top performer and also top goal score. Then standard deviation also showed for the above variables.

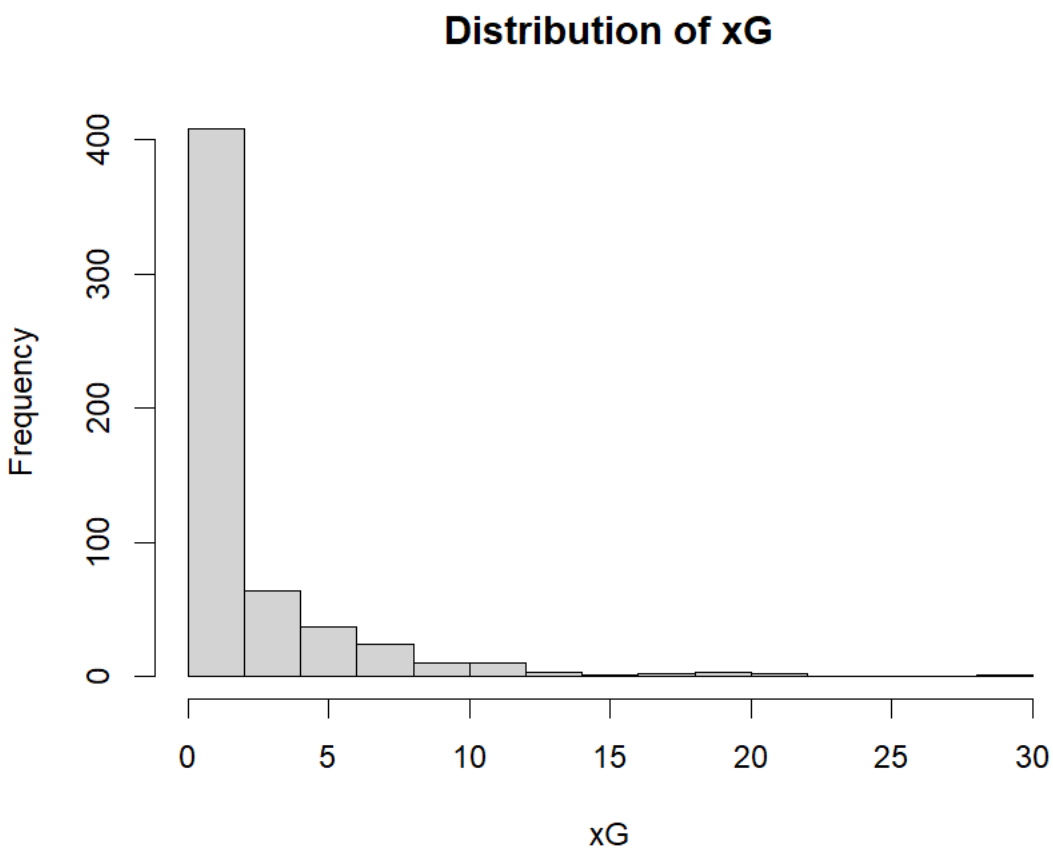
B. Distribution analysis:

Code: hist(pl$Gls, main="Distribution of Goals", xlab="Goals")



Interpret: the histogram of goals is right skewed which means most player scored few goals while few players scored many goals.

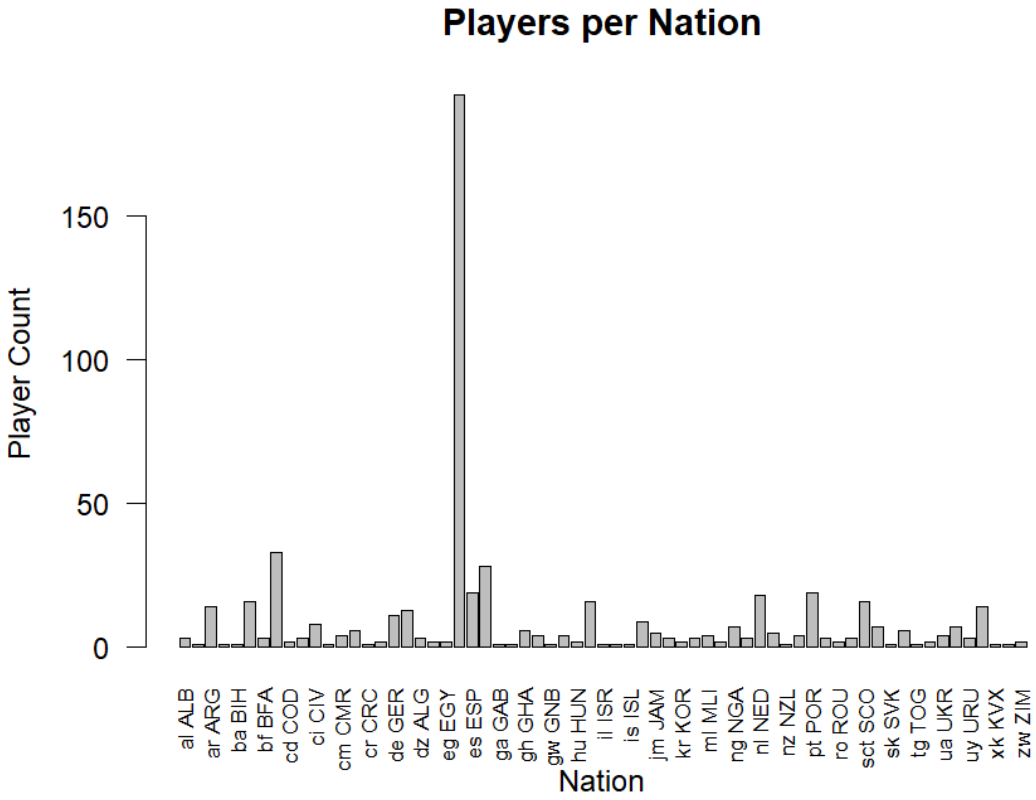
Code: hist(pl$xG, main="Distribution of xG", xlab="xG")



Interpret: the histogram of expected goals is also right skewed which indicates that expected scoring opportunities are unevenly distributed among players.

Code: nation_count <- table(pl$Nation)

barplot(nation_count,main = "Players per Nation",xlab = "Nation",ylab = "Player Count",las = 2,cex.names = 0.7)



Interpret: the bar plot showing that most of the premier league players are from Spain.

C. Comparison between groups:

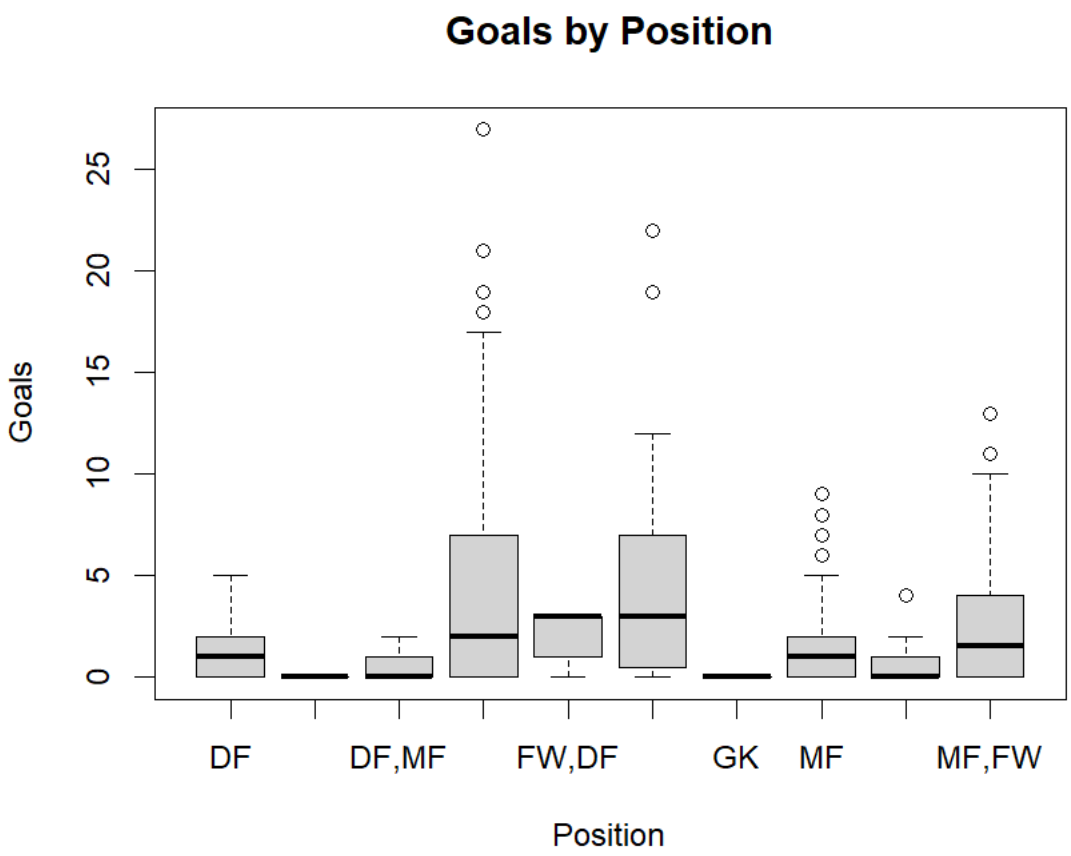
```
> aggregate(Gls ~ Pos, data = pl, mean)
     Pos       Gls
1     DF 0.9053254
2  DF,FW 0.0000000
3  DF,MF 0.4761905
4     FW 4.8241758
5  FW,DF 2.0000000
6  FW,MF 4.4181818
7     GK 0.0000000
8     MF 1.4324324
9  MF,DF 0.5625000
10 MF,FW 2.8541667
```

Interpret: this shows the comparison of average goal count between different positions. Like, Forwards have higher average goal count than midfielders and defenders.

```
> aggregate(xG ~ Pos, data = pl, mean)
     Pos        xG
1     DF 0.9159763
2  DF,FW 0.1555556
3  DF,MF 0.7380952
4     FW 4.9615385
5  FW,DF 2.6000000
6  FW,MF 3.9272727
7     GK 0.0000000
8     MF 1.4522523
9  MF,DF 1.1062500
10 MF,FW 2.8541667
```

Interpret: this shows the comparison of average expected goal count between different positions. Like, Forwards have higher average expected goal count than midfielders and defenders.

Code: boxplot(Gls ~ Pos, data=pl,main="Goals by Position",xlab="Position",ylab="Goals")



Interpret: the boxplot visualizes the previous information about the comparison of goals across different positions.

D. Variation analysis:

```
> aggregate(MP ~ Team, data=pl, sd)
                Team        MP
1            Arsenal 13.080902
2         Aston Villa 13.925116
3         Bournemouth 12.963952
4           Brentford 11.510308
5            Brighton  9.643906
6             Burnley 10.448835
7             Chelsea 11.809236
8      Crystal Palace 10.594048
9             Everton 13.055444
10             Fulham 11.583268
11          Liverpool 13.247598
12         Luton Town 11.153199
13    Manchester City 12.282824
14  Manchester United 11.412011
15   Newcastle United 11.735838
16  Nottingham Forest 10.706572
17   Sheffield United 11.602666
18  Tottenham Hotspur 12.614891
19    West Ham United 12.392119
20      Wolverhampton 12.816256
```

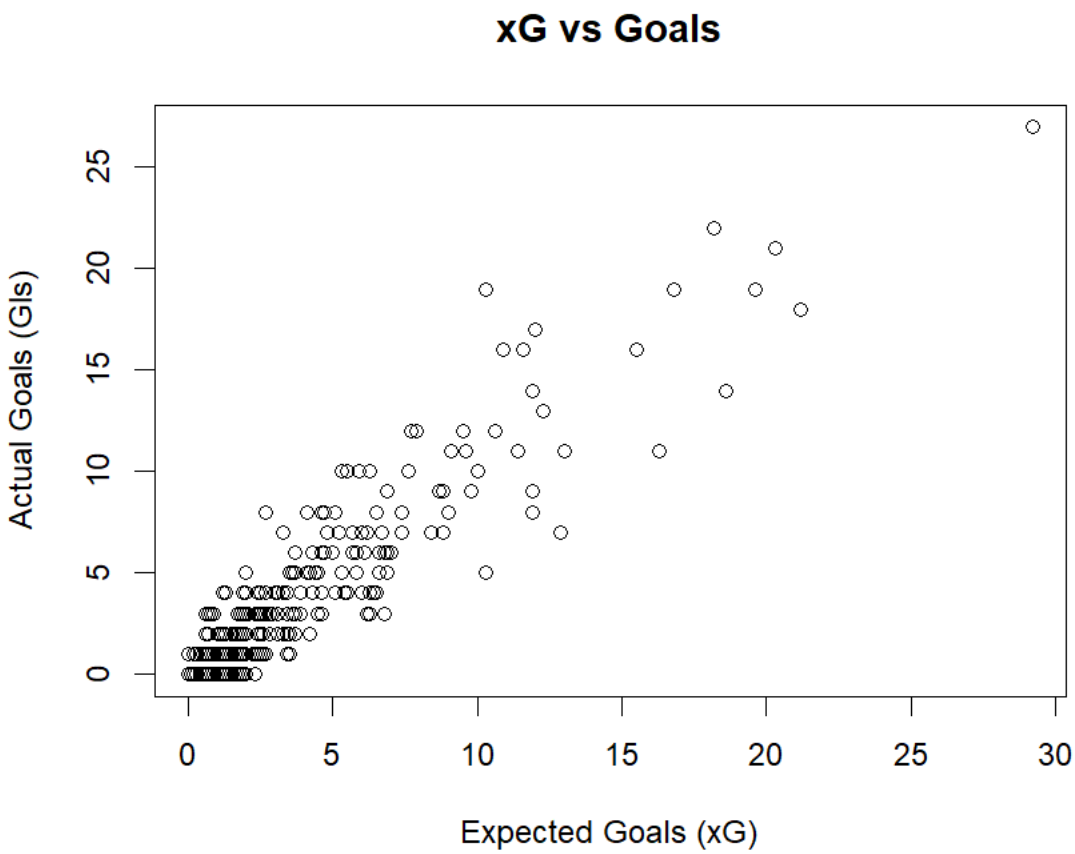
Interpret: this shows the standard deviation of minutes played across different teams. This analysis shows how player usage differs across teams.

E. Correlation:

```
> cor(pl[, c("Gls", "Ast", "xG", "xAG", "MP")])
          Gls       Ast        xG       xAG        MP
Gls 1.0000000 0.6028060 0.9355895 0.6510270 0.4970226
Ast 0.6028060 1.0000000 0.6017235 0.8970819 0.5271640
xG  0.9355895 0.6017235 1.0000000 0.6515769 0.5086662
xAG 0.6510270 0.8970819 0.6515769 1.0000000 0.5784108
MP  0.4970226 0.5271640 0.5086662 0.5784108 1.0000000
```

Interpret: A positive correlation exists between xG(expected goals) and Gls(actual goals) which indicates that players with higher expected scoring opportunities score more goals.

Code: plot(plxG, plGls,main="xG vs Goals",xlab="Expected Goals (xG)",ylab="Actual Goals (Gls)")



Interpret: this scatterplot shows the correlation between Gls and xG.

F. Insights:

```
> head(pl[order(pl$Gls, decreasing = TRUE), c("Player", "Team", "Gls")])
# A tibble: 6 × 3
  Player          Team               Gls
  <chr>           <chr>            <dbl>
1 Erling Haaland  Manchester City     27
2 Cole Palmer     Chelsea             22
3 Alexander Isak  Newcastle United    21
4 Phil Foden      Manchester City     19
5 Ollie Watkins   Aston Villa         19
6 Dominic Solanke Bournemouth         19
```

Interpret: This table shows the leading goal scorer and their team's name. which identifies the top performer of the league.

6. Train-Test split:

```
> set.seed(123)
> train_index <- sample(1:nrow(pl), 0.7 * nrow(pl))
> train <- pl[train_index, ]
> test  <- pl[-train_index, ]
> nrow(train)
[1] 395
> nrow(test)
[1] 170
> table(train$Pos)

   DF DF,FW DF,MF    FW FW,DF FW,MF    GK    MF MF,DF MF,FW
  119     8    15    60     5    39    26    78    10    35
> table(test$Pos)

   DF DF,FW DF,MF    FW FW,MF    GK    MF MF,DF MF,FW
   50     1     6    31    16    14    33     6    13
```

Interpret: the dataset was split into training set(70%) and testing set(30%). The training set contains 395 rows and testing set contains 170 rows. Also, the class distribution of the target variable (pos) is similar in both sets.

7. Conclusion:

In this project, we worked with a Premier League 2023/24 football dataset and carried out the full data science workflow, starting from cleaning and preparing the raw data to exploring it through descriptive statistics and visual analysis. We handled missing values, fixed incorrect and inconsistent entries, removed duplicates and normalized important attributes to make the data ready for analysis. Through various exploratory techniques, we showed distributions, compared groups, studied variations and identified meaningful patterns among players. Finally, split the cleaned dataset into training and testing sets to support future modeling.